

React Testing

Automated Testing

Automated tests are programs that automate the task of testing your software

Write code to test the software code

Additional effort is required when you develop a feature

Advantages

1. Not time-consuming
2. Reliable, consistent, and not error prone
3. Easy to identify and fix features that break tests
4. Gives confidence when shipping software

Jest and React Testing Library

Fundamentals of writing a test

Test components with user interactions

Test components wrapped in a provider

Test components with mocking

Static analysis testing

Jest

Jest is a JavaScript testing framework

Jest is a test runner that finds tests, runs the tests, determines whether the tests passed or failed, and reports it back in a human-readable manner

React Testing Library

JavaScript testing utility that provides virtual DOM for testing React components

React Testing Library provides a virtual DOM, which we can use to interact with and verify the behavior of a React component

Testing Library is, in fact, a family of packages that helps test UI components

The core library is called the DOM Testing library, and RTL is simply a wrapper around this core library to test React applications in an easier way

Types of tests

Unit tests

Integration tests

E2E tests

Unit tests

Focus is on testing the individual building blocks of an application, such as a class or a function or a component

Each unit or building block is tested in isolation, independent of other units

Dependencies are mocked

Run in a short amount of time and make it very easy to pinpoint failures

Relatively easier to write and maintain

Integration tests

Focus is on testing a combination of units and ensuring they work together

Take longer than unit tests

E2E tests

Focus is on testing the entire application flow and ensuring it works as designed from start to finish

Involves a real UI, a real backend database, real services, etc

Take the longest as they cover most amount of code

Have a cost implication as you interact with real APIs that may charge based on the number of requests

What is a Test?

So let's say we have a file, we write a test to check if the name is as expected or not

If it's not, then it will throw an error

```
const getFullName = (firstName, lastName) => {  
  return `${firstName} ${lastName}`;  
};  
  
console.log("running getFullName test...");  
  
const actualFullName = getFullName("John", "Doe");  
const expectedFullName = "John Do";  
  
if (actualFullName !== expectedFullName) {  
  throw new Error(  
    `Expected "${expectedFullName}", but got "${actualFullName}"`  
  );  
}  
  
//output:-  
//throw new Error(  
  ^  
//Error: Expected "John Do", but got "John Doe"  
//    at Object.<anonymous>
```

We already have React testing library and Jest in the node modules installed when we create a new React app

This is a basic test that checks if learn React is written on the page or not

Jest is a test runner that can find tests , run tests, and show user-friendly if the test passed or failed

So in scripts we have "react-scripts test"

which will internally call Jest

npm test is the command

Jest will run the test

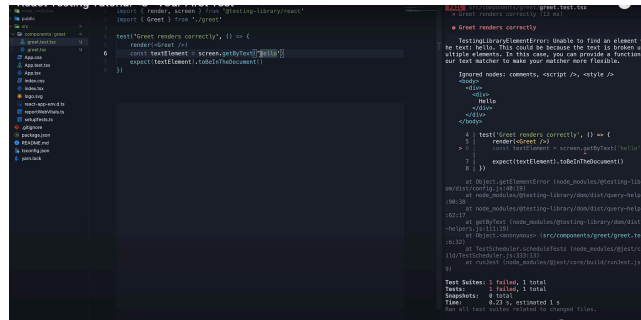
```
import React from "react";
import { render, screen } from "@testing-library/react";
import App from "./App";

//test method runs the test
//test(name,function,timeout)

//timeout is optional , tells how long to wait before
//aborting the test, default timeout value is 2 secs

test("renders learn react link", () => {
  render(<App />); // we start by creating a virtual DOM using render
  //method from @testing-library/react
  const linkElement = screen.getByText(/learn react/i);
  // we use screen which is object to query the virtual DOM
  expect(linkElement).toBeInTheDocument();
  // expectation is met here
});

//both test and expect are global functions
// we don't need to import them from anywhere
```



If the test is failing, then it gives an error message like this

Test Driven Development (TDD)

Test-driven development is a software development process where you write tests before writing the software code

Once the tests have been written, you then write the code to ensure the tests pass

1. Create tests that verify the functionality of a specific feature
2. Write software code that will run the tests successfully when re-executed
3. Refactor the code for optimization while ensuring the tests continue to pass

Also called red-green testing as all tests go from a red failed state to a green passed state

so here we write Test case description before writing test cases or component code

```
/**
 * Greet should render the test hello and if name is passed into
 * the component it should render the name as well.
 */
import { render, screen } from "@testing-library/react";
import React from "react";
import Greet from "./Greet";

test("Greet renders correctly", () => {
  render(<Greet />);
```

```

const textElement = screen.getByText(/hello/i);
expect(textElement).toBeInTheDocument();
});

test("Greet renders with a name", () => {
  render(<Greet name="John" />);
  const nameElement = screen.getByText(/john/i);
  expect(nameElement).toBeInTheDocument();
});

```

```

type GreetProps = {
  name?: string;
};

const Greet = (props: GreetProps) => {
  return <div>Hello {props.name}</div>;
};

export default Greet;

```

Jest Watch Mode

Watch mode is an option that we can pass to Jest, asking to watch files that have changed since the last commit and execute tests related only to those changed files

An optimization designed to make your tests run fast, regardless of how many tests you have

So when you do `npm test`, it will run only the tests that have been changed or updated

Filtering Tests

When we run `npm test` in terminal, we have options like `-a` (display all tests), `-p` (display test based on pattern) and so on

We can use `test.only` or `test.skip` to filter tests

Grouping Tests

`describe(name, fn)`

The first argument is the group name

The second argument is a function that contains the expectations to test

```
import { render, screen } from "@testing-library/react";
import React from "react";
import Greet from "./Greet";

describe.only("Greet", () => {
  test("renders correctly", () => {
    render(<Greet />);
    const textElement = screen.getByText(/hello/i);
    expect(textElement).toBeInTheDocument();
  });

  describe("Nested", () => {
    test("renders with a name", () => {
      render(<Greet name="John" />);
      const nameElement = screen.getByText(/john/i);
      expect(nameElement).toBeInTheDocument();
    });
  });
});
```

`Describe.only`

- Runs **only** that specific test suite.
- All other `describe` blocks in the test file (or even across files) are ignored.
- Useful when debugging or working on one specific group of tests.

Describe.skip

- Skips that test suite completely.
- Useful if the test is flaky, failing, or not relevant for now.

One file is one test suites

Filename Conventions

Files with test.js or .test.tsx suffix

Files with .spec.js or .spec.tsx suffix

Files with .js or .tsx suffix in **__tests__** folders

The recommendation is to always put your tests next to the code they are testing so that relative imports are shorter.

```
import { render, screen } from '@testing-library/react'
import { Greet } from "../greet"

describe('Greet', () => {
  it('renders correctly', () => {
    render(<Greet />)
    const textElement = screen.getByText('Hello')
    expect(textElement).toBeInTheDocument()
  })
})

describe('Nested', () => {
  it('renders a name', () => {
    render(<Greet name='Vishwas' />)
    const textElement = screen.getByText('Hello Vishwas')
    expect(textElement).toBeInTheDocument()
  })
})
})
```

An alternative to the test method is it

We can use it, fit → test.only and xit → skip test

Code Coverage

A metric that can help you understand how much of your software code is tested

1. Statement coverage: How many of the statements in the software code have been executed
2. Branches coverage: How many of the branches of the control structures (if statements, for instance) have been executed
3. Function coverage: How many of the functions defined have been called
4. Line coverage: How many lines of source code have been tested

We add this line

```
"scripts": {  
  "start": "react-scripts start",  
  "build": "react-scripts build",  
  "test": "react-scripts test",  
  "eject": "react-scripts eject",  
  "coverage": "react-scripts test --coverage"  
},
```

Then we do npm run coverage

```
> my-app@0.1.0 coverage  
> react-scripts test --coverage  
PASS src/components/greet/Greet.test.jsx  
PASS src/components/greet/test.jsx  
PASS src/components/greet/Greet.test.jsx  
PASS src/App.test.jsx  
-----  
File | % Stats | % Branch | % Funcs | % Lines | Uncovered Line #s  
-----  
All files | 100 | 100 | 100 | 100 |  
src | 100 | 100 | 100 | 100 |  
  App.jsx | 100 | 100 | 100 | 100 |  
  src/components/greet | 100 | 100 | 100 | 100 |  
    Greet.jsx | 100 | 100 | 100 | 100 |  
-----  
Test Suites: 2 passed, 2 total  
Tests: 3 passed, 3 total  
Snapshots: 0 total  
Time: 1.86 s, estimated 2 s  
Ran all test suites related to changed files.
```

If we add --watchAll

```
PASS src/components/Greet/Greet.test.tsx
PASS src/App.test.tsx
PASS src/App.test.tsx
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	21.42	0	50	21.42	
src	8.33	0	33.33	8.33	
App.tsx	100	100	100	100	
index.tsx	0	100	100	0	7-19
reportWebVitals.ts	0	0	0	0	3-10
src/components/Greet	100	100	100	100	
Greet.tsx	100	100	100	100	

```
Test Suites: 2 passed, 2 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 2.438 s
Ran all test suites.
```

But index.tsx and reportWebVitals are not important

So we will ignore them

```
"coverage": "react-scripts test --coverage --watchAll --collectCoverageFrom
='src/components/**/*.{js,jsx,ts,tsx}'"
```

To ignore files, we can use (when dealing with larger application,s we are bound to have files that don't need testing)

so if I have file name greet.types.ts

```
collectCoverageFrom = "!src/components/**/*.{types,stories,constants,test,spec}.
{ts,tsx}'
```

To test Threshold jest will report error if we have less than 80 % code coverage

```
"jest": {
  "coverageThreshold": {
    "global": {
      "branches": 80,
      "functions": 80,
      "lines": 80,
      "statements": -10
    }
  }
}
```

Assertions

When writing tests, we often need to check that values meet certain conditions

Assertions decide if a test passes or fails

Jest assertions are carried out with the global expect method

`expect(value)` → The argument should be the value that your code produces

Typically, you will use expect along with a “matcher” function to assert something about a value

A matcher can optionally accept an argument, which is the correct expected value

You can view matchers here, but they are for JS code only and not for UI

<https://jestjs.io/docs/using-matchers>

for UI, we have Jest-dom using which we can use matchers like

`expect(textElement).toBeInTheDocument();`

<https://github.com/testing-library/jest-dom>

It is a global setup file that is why we don't have to import it

What to test?

1. Test if the component is rendering
2. Test if the component renders the right props
3. Test if the component renders in different states
4. Test if the component reacts to events

What not to test → Implementation details, Third-party code (No need to test MUI button) , Code that is not important from a user point of view (if a function displays

a date in specific format no need to worry if function is getting called or not, just test if the format of the date that is displayed is correct or not)

RTL Queries

Every test we write generally involves the following basic steps

1. Render the component
2. Find an element rendered by the component
3. Assert against the element found in step 2, which will pass or fail the test

To render the component, we use the render method from RTL

For assertion, we use expect passing in a value and combine it with a matcher function from jest or jest-dom

Queries are the methods that Testing Library provides to find elements on the page

To find a single element on the page, we have

- getBy..
- queryBy..
- findBy..

To find multiple elements on the page, we have

- getAllBy..
- queryAllBy..
- findAllBy..

The suffix can be one of Role, LabelText, PlaceholderText, Text, DisplayValue, AltText, Title and finally TestId

getBy.. queries

Returns the matching node for a query, and throws a descriptive error if no elements match or if more than one match is found

getByRole

getByRole queries for elements with given role

Role refers to ARIA (Accessible Rich Internet Applications) role which provides semantic meaning to content to ensure people using assistive technologies are able to use them

By default, many semantic elements in HTML have a role

Button element has a button role, anchor element has a link role, h1 to h6 elements have a heading role, checkboxes have a checkbox role, radio buttons have a radio role and so on

If you're working with elements that do not have a default role or if you want to specify a different role, the role attribute can be used to add the desired role

To use an anchor element as a button in the navbar, you can add role="button"

```
export const Application = () => {
  return (
    <>
      <h1>Job application form</h1>
      <h2>Section 1</h2>
      <p>All fields are mandatory</p>
      <span title="close">X</span>
      
    <div data-testid="custom-element">Custom HTML element</div>
    <form>
      <div>
        <label htmlFor="name">Name</label>
        <input type="text" id="name" />
      </div>
      <div>
```

```

    //<label htmlFor="bio">Bio</label>
    //<textarea id="bio" />
  </div>
  <div>
    <label htmlFor="job-location">Job location</label>
    <select id="job-location">
      <option value="">Select a country</option>
      <option value="US">United States</option>
      <option value="GB">United Kingdom</option>
      <option value="CA">Canada</option>
      <option value="IN">India</option>
      <option value="AU">Australia</option>
    </select>
  </div>
  <div>
    <label>
      <input type="checkbox" id="terms" /> I agree to the terms and
      conditions
    </label>
  </div>
  <button>Submit</button>
</form>
</>
);
};

```

```

import { render, screen } from "@testing-library/react";
import React from "react";
import { Application } from "../Applications";

describe("Application", () => {
  test("renders correctly", () => {
    render(<Application />);
    const input = screen.getByRole("textbox");
    expect(input).toBeInTheDocument();
  });
});

```

```
const jobLocationSelect = screen.getByRole("combobox");
expect(jobLocationSelect).toBeInTheDocument();

const termsElement = screen.getByRole("checkbox");
expect(termsElement).toBeInTheDocument();

const submitButton = screen.getByRole("button", { name: /submit/i });
expect(submitButton).toBeInTheDocument();
});
});
```

<https://testing-library.com/docs/queries/byrole>

<https://testing-library.com/docs/queries/byrole>

<https://www.w3.org/TR/html-aria/#docconformance>

To know the roles of HTML elements

If you get multiple textbox, we can use options

name → The accessible name is, for simple cases, equal to

1. The label of a form element
2. The text content of a button or
3. The value of the aria-label attribute

```
const input = screen.getByRole("textbox", {
  name: /name/i,
});
expect(input).toBeInTheDocument();
```

```
const headingElement = screen.getByRole("heading", {
  name: /job application form/i,
});

expect(headingElement).toBeInTheDocument();

const sectionElement = screen.getByRole("heading", {
  name: /section 1/i,
});
expect(sectionElement).toBeInTheDocument();

//if we want to ensure that heading is h1 or h2 we have a level option

const headingElement = screen.getByRole("heading", {
  level: 1,
});

expect(headingElement).toBeInTheDocument();

const sectionElement = screen.getByRole("heading", {
  level: 2,
});
expect(sectionElement).toBeInTheDocument();
```

getByLabelText

getByLabelText will search for the label that matches the given text, then find the element associated with that label


```
<label htmlFor="name">Name</label>
  <input type="text" id="name" />
```

// will find the label and get input associated with that label

```
const nameElement = screen.getByLabelText("Name");
expect(nameElement).toBeInTheDocument();
```

//also works for wrapped elements

```
<label>
  <input type="checkbox" id="terms" /> I agree to the terms and
  conditions
</label>
```

```
const termsElement = screen.getAllByLabelText("I agree to the terms and c
onditions");
expect(termsElement).toBeInTheDocument();
```

// if multiple values found then we can use this

```
const nameElement = screen.getByLabelText("Name", { selector: "input"
});
expect(nameElement).toBeInTheDocument();
```

getByPlaceholderText

getByPlaceholderText will search for all elements with a placeholder attribute and find one that matches the given text

```
<label htmlFor="name">Name</label>
  <input type="text" id="name" placeholder="Fullname" />
```

```
const nameElementWithPlaceholder = screen.getByPlaceholderText("Fullna
```

```
me");  
expect(nameElementWithPlaceholder).toBeInTheDocument();
```

getByText

getByText will search for all elements that have a text node with textContent matching the given text

Typically, we use this to find paragraph, div, or span elements

```
<p>All fields are mandatory</p>  
  
const paragraphElement = screen.getByText(/all fields are mandatory/i);  
expect(paragraphElement).toBeInTheDocument();
```

getByDisplayValue

getByDisplayValue returns the input, textarea, or select element that has the matching display value

```
<form>  
  <div>  
    <label htmlFor="name">Name</label>  
    <input  
      type="text"  
      id="name"  
      placeholder="Fullname"  
      value="John"  
      onChange={() => {}}  
    />  
  </div>  
</form>  
  
const nameElement = screen.getByDisplayValue('John')
```

```
expect(nameElement).toBeInTheDocument();
```

getByAltText

getByAltText will return the element that has the given alt text

```

```

// we can use this or

```
const imageElement = screen.getByRole("img", {  
  name: /a person with a laptop/i,  
});  
expect(imageElement).toBeInTheDocument();
```

// we can use this

```
const imageElement = screen.getByAltText("a person with a laptop");  
expect(imageElement).toBeInTheDocument();
```

getByTitle

getByTitle returns the element that has the matching title attribute

```
<span title="close">X</span>
```

```
const closeElement = screen.getByTitle("close");  
expect(closeElement).toBeInTheDocument();
```

getByTestId

getByTestId returns the element that has the matching data-testid attribute

```
<div data-testid="custom-element">Custom HTML element</div>

const customElement = screen.getByTestId('custom-element')
expect(customElement).toBeInTheDocument()
```

Priority Order for Queries

Your test should resemble how users interact with your code (component,page,etc) as much as possible

Order →

1. getByRole
2. getByLabelText
3. getByPlaceholderText
4. getByText
5. getByDisplayValue
6. getByAltText
7. getByTitle
8. getByTestId

Query Multiple Elements

getAllBy Queries returns an array of all matching nodes for a query, and throws an error if no elements match

```
import { useState, useEffect } from "react";
import { SkillsProps } from "../Skills.types";

export const Skills = (props: SkillsProps) => {
```

```

const { skills } = props;
const [isLoggedIn, setIsLoggedIn] = useState(false);

useEffect(() => {
  setTimeout(() => {
    setIsLoggedIn(true);
  }, 1001);
}, []);
return (
  <>
    <ul>
      {skills.map((skill) => {
        return <li key={skill}>{skill}</li>;
      })}
    </ul>
    {isLoggedIn ? (
      <button>Start learning</button>
    ) : (
      <button onClick={() => setIsLoggedIn(true)}>Login</button>
    )}
  </>
);
};

```

```

import React from "react";
import { render, screen } from "@testing-library/react";
import { Skills } from "./Skills";

describe("Skills", () => {
  const skills = ["JavaScript", "React", "Node.js"];
  test("renders correctly", () => {
    render(<Skills skills={skills} />);
    const listElement = screen.getByRole("list");
    // console.log("check value", listElement);
  });
});

```

```

    expect(listElement).toBeInTheDocument();
  });

  test("displays the correct number of skills", () => {
    render(<Skills skills={skills} />);
    const skillItems = screen.getAllByRole("listitem");
    expect(skillItems).toHaveLength(skills.length);
  });
});

```

TextMatch

TextMatch represents a type which can be either a string, a regex, or a function

TextMatch - string

```

<div>Hello World</div>

screen.getByText('Hello World') // full string match
screen.getByText('llo Worl', {exact: false}) // substring match
screen.getByText('hello world', {exact: false}) // ignore case

```

TextMatch - regex

```

<div>Hello World</div>

screen.getByText(/World/) // substring match
screen.getByText(/world/i) // substring match, ignore case
screen.getByText(/^hello world$/i) // full string match, ignore case

```

TextMatch - custom function

```
(content?: string, element?: Element | null) => boolean  
  
<div>Hello World</div>  
  
screen.getByText((content) => content.startsWith('Hello'))
```

queryBy

Returns the matching node for a query and return null if no elements match

Useful for asserting an element that is not present

Throws an error if more than one match is found

queryAllBy

Returns an array of all matching nodes for a query, and return an empty array if no elements match

Useful for asserting an element that is not present in the DOM

So if we want to write a test for an element that does not exist in the DOM we can use queryBy

We make changes to the code by adding a isLoggedIn button logic

```
import { useState, useEffect } from 'react'  
import { SkillsProps } from './Skills.types'  
  
export const Skills = (props: SkillsProps) => {  
  const { skills } = props  
  const [isLoggedIn, setIsLoggedIn] = useState(false)  
  
  useEffect(() => {
```

```

    setTimeout(() => {
      setIsLoggedIn(true)
    }, 1001)
  }, [])
  return (
    <>
      <ul>
        {skills.map((skill) => {
          return <li key={skill}>{skill}</li>
        })}
      </ul>
      {isLoggedIn ? (
        <button>Start learning</button>
      ) : (
        <button onClick={() => setIsLoggedIn(true)}>Login</button>
      )}
    </>
  )
}

```

```

test("start learning button is not displayed initially", () => {
  render(<Skills skills={skills} />);
  // this will wait for the state to update after the timeout in useEffect
  // if we use findByRole it will throw an error if the element is not found
  const startLearningButton = screen.queryByRole("button", {
    name: /start learning/i,
  });
  // returns null if the element is not found
  console.log("startLearningButton", startLearningButton);
  expect(startLearningButton).not.toBeInTheDocument();
});
});

```


findBy

What if elements are not present in the DOM to begin but make their way into the DOM after some time

For example, data that is fetched from a server will be rendered only after a few milliseconds

In the previous code, we have `useEffect` after 5 seconds, `set` timeout will change the state of `isLoggedIn`

`getByRole` does not wait, also `queryBy` doesn't wait, but `findBy` does wait

findBy

Returns a Promise which resolves when an element is found which matches the given query

The promise is rejected if no element is found or if more than one element is found after a default timeout of 1000 ms

findAllBy

Returns a promise which resolves to an array of elements when any elements are found which matches the given query

The promise is rejected if no elements are found after default timeout of 1000ms

```
useEffect(() => {  
  setTimeout(() => {  
    setIsLoggedIn(true);  
  }, 500);  
}, []);
```

```
test("start learning button is eventually displayed ", async () => {  
  render(<Skills skills={skills} />);  
  const startLearningButton = await screen.findByRole(  

```

```

    "button",
    {
      name: /start learning/i,
    },
    {
      timeout: 2000, // wait for up to 1 second
    }
  );
  expect(startLearningButton).toBeInTheDocument();
});

```

Manual Queries

We can also use regular `querySelector` DOM api to find elements, but it's not recommended

When testing (especially in React Testing Library, Jest DOM, Cypress, etc.), the main reason we avoid **normal query selectors** like `document.querySelector()` or `getElementById()` is because they test **implementation details**, not the **user experience**.

Here's why that's a problem:

1. Tests become tightly coupled to the DOM structure

- `querySelector` directly targets specific HTML tags or class names.
- If you change markup (e.g., from `<div>` to `<span>`, or rename a class), the test will fail even if the actual behavior for the user is unchanged.
- Good tests should still pass after safe refactors.

2. It ignores how a real user interacts

- Real users don't see HTML tags or CSS classes — they see text, labels, roles, and placeholders.
- React Testing Library promotes using queries like:

- `getByText()` → simulates how a user sees text.
- `getByRole()` → simulates accessibility navigation.
- `getByLabelText()` → simulates how a user finds a form field.

3. It reduces accessibility checks

- `querySelector` doesn't care if your app has correct ARIA roles, labels, or alt text.
- Using `getByRole()` or `getByLabelText()` indirectly enforces accessibility best practices.

4. It makes tests fragile during styling changes

- If you change CSS classes, `querySelector('.btn-primary')` breaks.
- But if you test with `getByRole('button', { name: /submit/i })`, the test still passes.

5. It leads to testing the "how" instead of the "what"

- With `querySelector`, you're checking **how** the DOM is structured.
- With RTL queries, you check **what** the user actually experiences.

Debugging

We have a debug option

```
components > skills > skills.test.tsx > describe("Skills") callback > test("Start learning button is eventually displayed") callback
const loginButton = screen.getByRole("button", {
  name: "Login",
});
expect(loginButton).toBeInTheDocument();

test("Start learning button is not rendered", () => {
  render(<Skills skills={skills} />);
  const startLearningButton = screen.queryByRole("button", {
    name: "Start learning",
  });
  expect(startLearningButton).not.toBeInTheDocument();
});

test("Start learning button is eventually displayed", async () => {
  render(<Skills skills={skills} />);
  screen.debug();
  const startLearningButton = await screen.findByRole(
    "button",
    {
      name: "Start learning",
    },
    {
      timeout: 2000,
    }
  );
  screen.debug();
  expect(startLearningButton).toBeInTheDocument();
});
});
```

```
<div>
  <ul>
    <li>
      HTML
    </li>
    <li>
      CSS
    </li>
    <li>
      JavaScript
    </li>
  </ul>
  <button>
    Login
  </button>
</div>
</body>

/Users/vishwas/Documents/Codevolution/Course/React Testing
/react-testing/src/components/skills/skills.test.tsx:37:12
35 |   test("Start learning button is eventually display
    |   ^
36 |     render(<Skills skills={skills} />);
37 |     screen.debug();
    |     ^
    at logDOM (node_modules/@testing-library/dom/dist/pretty
-dom.js:88:13)

console.log
  <body>
    <div>
      <ul>
        <li>
          HTML
        </li>
        <li>
          CSS
        </li>
        <li>
          JavaScript
        </li>
      </ul>
      <button>
        Start learning
      </button>
    </div>
  </body>
```

```
App.tsx skills.types.ts skills.tsx M skills.test.tsx X application.test.tsx
src > components > skills > skills.test.tsx > describe("Skills") callback > test("Start learning button is eventually displayed") callback
29 const startLearningButton = screen.queryByRole("button", {
30   name: "Start learning",
31 });
32 expect(startLearningButton).not.toBeInTheDocument();
33 });
34
35 test("Start learning button is eventually displayed", async () => {
36   const view = render(<Skills skills={skills} />);
37   logRoles(view.container);
38   // screen.debug();
39   const startLearningButton = await screen.findByRole(
40     "button",
41     {
42       name: "Start learning",
43     },
44     {
45       timeout: 2000,
46     }
47   );
48   // screen.debug();
49   expect(startLearningButton).toBeInTheDocument();
50 });
51 });
52
```

```
console.log
list:
  Name "":
  <ul />

  listitem:
    Name "":
    <li />

    Name "":
    <li />

    Name "":
    <li />

  button:
    Name "Login":
    <button />

at logRoles (node_modules/@testing-library/dom/dist/role
helpers.js:234:20)

PASS src/components/skills/skills.test.tsx
Skills
  ✓ renders correctly (30 ms)
  ✓ renders a list of skills (11 ms)
  ✓ renders Login button (9 ms)
  ✓ Start learning button is not rendered (3 ms)
  ✓ Start learning button is eventually displayed (1012 ms)

Test Suites: 1 passed, 1 total
Tests: 5 passed, 5 total
Snapshots: 0 total
Time: 1.888 s, estimated 2 s
Ran all test suites related to changed files.

Watch Usage: Press w to show more.
```

using debug we can get formatted dom tree at any point of test and using logRoles we can get list of aria roles present in the dom tree

What does an ARIA role do?

- It gives a semantic meaning to an element, even if the HTML element itself doesn't have it by default.
- Example:

```
html
CopyEdit
<div role="button">Click me</div>
```

A `<div>` normally isn't a button, but with `role="button"`, a screen reader will treat it like one.

Testing Playground

The screenshot displays the Testing Playground interface. On the left, a web application titled "Job application form" is shown. It includes a "Section 1" header, a note "All fields are mandatory", a 150x150 placeholder image, a "Custom HTML element" label, a "Name" input field with the value "Vishwas", a "Bio" input field, a "Select a country" dropdown menu, a checkbox for "I agree to the terms and conditions", a "Submit" button, and a "Start learning" button. On the right, the "Testing Playground" panel is open, showing a list of accessible queries. The "Role" query is selected, displaying the value "button". Other queries include "LabelText", "PlaceholderText", "Text", "DisplayValue", "AriaText", "Title", "Testid", and "DisplayValue". A "suggested query" section at the top right shows a query for "button" with the name "/submit/1".

User Interactions

A Click using a mouse or a keypress using a keyboard

Software has to respond to such interactions

Tests should ensure the interactions are handled as expected

User-event → it's a library that simulates user interactions by dispatching the events that would happen if the interaction took place in a browser

It is the recommended way to test user interactions with RTL

There is also an alternative fireEvent

fireEvent vs user-event

fireEvent is a method from RTL which is used to dispatch DOM events

user-event simulates full interactions, which may fire multiple events and do additional checks along the way

For example, we can dispatch the change event on an input field using fireEvent

But when a user types into text box, the element has to be focused, and then keyboard and input events are fired and the selection and value on the element are manipulated as they type

user-event allows you to describe a user interaction instead of a concrete event. It adds visibility and intractability checks along the way and manipulates the DOM just like a user interaction in the browser would. It factors in that the browser e.g wouldn't let a user click a hidden element or type in a disabled text box

Pointer Interactions

Let's say we have this code

```
import { useState } from 'react'
```

```

export const Counter = () => {
  const [count, setCount] = useState(0)
  const [amount, setAmount] = useState(0)
  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <input
        type="number"
        name="amount"
        value={amount}
        onChange={(e) => setAmount(parseInt(e.target.value))}
      />
      <button onClick={() => setCount(amount)}>Set</button>
    </div>
  )
}

```

```

import { render, screen } from "@testing-library/react";
import React from "react";
import { Counter } from "../Counter";
import user from "@testing-library/user-event";

describe("Counter", () => {
  test("renders correctly", () => {
    render(<Counter />);
    const countElement = screen.getByRole("heading");
    expect(countElement).toBeInTheDocument();

    const incrementButton = screen.getByRole("button", { name: /increment/i });
    expect(incrementButton).toBeInTheDocument();
  });
});

```

```

test("initial count is 0", () => {
  render(<Counter />);
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("0");
});

test("render a count of 1 after clicking increment button", async () => {
  // user.setup();
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i
});
  await user.click(incrementButton);
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("1");
});

test("render a count of 5 after clicking increment button", async () => {
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i
});
  for (let i = 0; i < 5; i++) {
    await user.click(incrementButton);
  }
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("5");
});

test("render a count of 2 after clicking increment button", async () => {
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i
});
  await user.dblClick(incrementButton);
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("2");
});

```



```
});  
});
```

Click is not a pointer API →

It is Convenience APIs which internally calls pointer api

we have click() , dblClick() , tripleClick() , hover() , unhover()

Pointer APIs

pointer accepts object as argument

pointer({keys:'[MouseLeft]'})

pointer({keys:'[MouseLeft][Mouseright]'})

also pass in string if keys is the only argument to the function

pointer('[MouseLeft][Mouseright]')

Convenience APIs

- **What they are:**

Pre-built helper functions or abstractions that make a developer's job easier by avoiding repetitive, low-level code.

- **Why they exist:**

Instead of manually wiring up all the details, you can call a "convenience" method that handles them for you.

- **Example contexts:**

- **Web Testing (React Testing Library):**

`screen.getByRole('button', { name: 'Submit' })` → convenience over manually querying DOM elements by selectors.

- **OS Accessibility APIs:**

`UIElement.Click()` might directly trigger a click action instead of having to simulate pointer down + pointer up events yourself.

- **Key point:**

It's not a special *type* of API like "Pointer API" — it's just a *category* meaning "easier-to-use shortcut APIs."

Pointer APIs

- **What they are:**

APIs that handle *pointing device input* — any input where a user can move a pointer (mouse, trackpad, touchscreen, pen).

- **Purpose:**

Give developers a unified way to respond to pointer actions like move, click, drag, pressure, and multi-touch.

- **Example (Web Pointer Events API):**

```
element.addEventListener('pointerdown', (event) => {  
  console.log(event.pointerType); // "mouse" | "touch" | "pen"  
});
```

- **Benefits:**

- Works for mouse, touch, and pen without separate logic.
- Gives richer data than mouse-only APIs (e.g., pressure, tilt, multi-pointer).



Simple difference:

- **Convenience API** → High-level helper method that hides complexity.
- **Pointer API** → Low-level (but standardized) interface for pointer-based input devices.

Let's take your `"render a count of 1 after clicking increment button"` test and rewrite it without `user.click()`, using **raw Pointer Events**.

Original (Convenience API via `user.click`)

```
test("render a count of 1 after clicking increment button", async () => {
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i });
  await user.click(incrementButton); // convenience API
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("1");
});
```

Rewritten (Manual Pointer API dispatch)

```
test("render a count of 1 after clicking increment button", () => {
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i });

  // Simulate pointerdown → pointerup → click
  incrementButton.dispatchEvent(
    new PointerEvent("pointerdown", { bubbles: true })
  );
  incrementButton.dispatchEvent(
    new PointerEvent("pointerup", { bubbles: true })
  );
  incrementButton.dispatchEvent(
    new MouseEvent("click", { bubbles: true })
  );

  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("1");
});
```

What's the difference?

Method	Type	What happens
<code>user.click()</code>	Convenience API	Automatically fires the correct pointer + mouse events in the right sequence, handles delays, focuses the element, and matches browser behavior.
<code>dispatchEvent(new PointerEvent(...))</code>	Pointer API	You manually fire the raw events, one by one, with all required options.

Keyboard interactions

So we have this code

```
import { useState } from 'react'

export const Counter = () => {
  const [count, setCount] = useState(0)
  const [amount, setAmount] = useState(0)
  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <input
        type="number"
        name="amount"
        aria-label="Amount"
        value={amount}
        onChange={(e) => setAmount(parseInt(e.target.value))}
      />
      <button onClick={() => setCount(amount)}>Set</button>
    </div>
  )
}
```

So we want to test if the amount is set correctly or not, we simulate these interactions

```
test("render count of 10 after setting amount to 10 and clicking set button", a
sync () => {
  render(<Counter />);
  const amountInput = screen.getByRole("spinbutton", { name: /amount/i });
  await user.type(amountInput, "10");
  expect(amountInput).toHaveValue(10);
  const setButton = screen.getByRole("button", { name: /set/i });
  await user.click(setButton);
  const countElement = screen.getByRole("heading");
  expect(countElement).toHaveTextContent("10");
});
```

Now we want to check If clicking on the tab button on the keyboard will the focus on inputs shift properly or not

```
test("elements are focused correctly", async () => {
  render(<Counter />);
  const incrementButton = screen.getByRole("button", { name: /increment/i
});
  const amountInput = screen.getByRole("spinbutton", { name: /amount/i });
  const setButton = screen.getByRole("button", { name: /set/i });

  await user.tab();
  expect(incrementButton).toHaveFocus();

  await user.tab();
  expect(amountInput).toHaveFocus();

  await user.tab();
```

```
expect(setButton).toHaveFocus();  
});
```

type(),clear() ,selectOptions(),deselectOptions() → is utility API

tab()→ convenience API

copy(), cut(), paste() → clipboard APIs

keyboard APIs

Providers

To test providers, we will test theme MUI

Let's say we have this App provider code

```
import { ThemeProvider, createTheme } from '@mui/material/styles'  
import CssBaseline from '@mui/material/CssBaseline'  
  
const theme = createTheme({  
  palette: {  
    mode: 'dark', // can change mode here  
  },  
})  
  
export const AppProviders = ({ children }: { children: React.ReactNode }) => {  
  return (  
    <ThemeProvider theme={theme}>  
      <CssBaseline />  
      {children}  
    </ThemeProvider>  
  )  
}
```

We have MUI mode

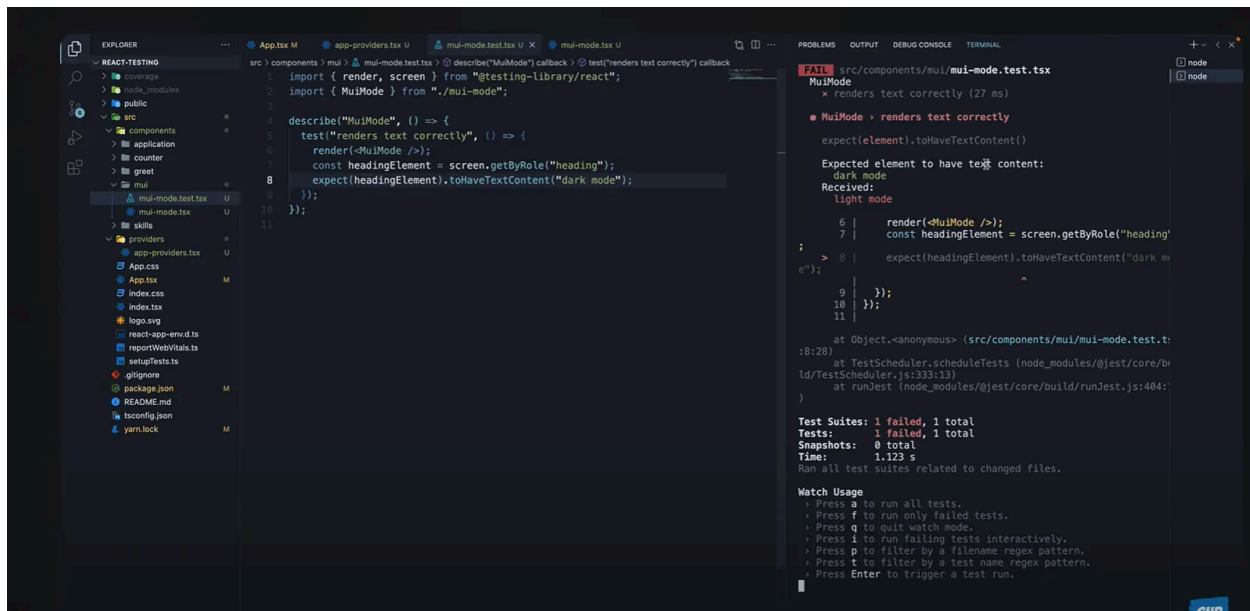
```
import { useTheme } from '@mui/material/styles'
import { Typography } from '@mui/material'

export const MuiMode = () => {
  const theme = useTheme()
  return (
    <>
      <Typography component="h1">`${theme.palette.mode} mode`</Typogr
aphy>
    </>
  )
}
```

```
import './App.css'
import { MuiMode } from './components/mui/MuiMode'
function App() {
  return (
    <AppProviders>
      <div className="App">
        <MuiMode />
      </div>
    </AppProviders>
  )
}

export default App
```

without using wrapper in render method test will fail



```
render(<MuiMode />, { wrapper: AppProviders });
```

We we dont want to add wrapper everytime we can use custom render function

Custom React Hooks

If we want to test custom hooks

we can just call custom hook function in render method it throws error

custom hook

```
import { useState } from 'react'
import { UseCounterProps } from './userCounner.types'

export const useCounter = ({ initialCount = 0 }: UseCounterProps = {}) => {
  const [count, setCount] = useState(initialCount)
  const increment = () => setCount(count + 1)
  const decrement = () => setCount(count - 1)
  return { count, increment, decrement }
}
```


We can use render hook method for this returns object and we can extract result returned from it

```
import { renderHook, act } from '@testing-library/react'
import { useCounter } from './useCounter'

describe('useCounter', () => {
  test('should render the initial count', () => {
    const { result } = renderHook(useCounter)
    expect(result.current.count).toBe(0)
  })
})
```

If we want to pass initial props and test if it's rendering properly or not, we can add an extra parameter to render hook method

```
test('should accept and render the same initial count', () => {
  const { result } = renderHook(useCounter, {
    initialProps: { initialCount: 10 },
  })
  expect(result.current.count).toBe(10)
})
```

Act Utility

so in custom hooks when we want to test increment method or state change we need to wrap it in Act

```
test('should increment the count', () => {
  const { result } = renderHook(useCounter)
  act(() => result.current.increment())
  expect(result.current.count).toBe(1)
})

test('should decrement the count', () => {
```

```
const { result } = renderHook(useCounter)
act(() => result.current.decrement())
expect(result.current.count).toBe(-1)
})
```

Previously, it was a button click then it would increment but when state updates or react component re-renders we use act

Mocking Functions

To test functions that are passed to components as props we can follow this process

counter component

```
import { CounterTwoProps } from './CounterTwo.types'

export const CounterTwo = (props: CounterTwoProps) => {
  return (
    <div>
      <h1>Counter Two</h1>
      <p>{props.count}</p>
      {props.handleIncrement && (
        <button onClick={props.handleIncrement}>Increment</button>
      )}
      {props.handleDecrement && (
        <button onClick={props.handleDecrement}>Decrement</button>
      )}
    </div>
  )
}
```

jest.fn() will create mock functions and these functions can be passed as props in render method

```

import { render, screen } from '@testing-library/react'
import user from '@testing-library/user-event'
import { CounterTwo } from './CounterTwo'

test('renders correctly', () => {
  render(<CounterTwo count={0} />)
  const textElement = screen.getByText('Counter Two')
  expect(textElement).toBeInTheDocument()
})

test('handlers are called', async () => {
  user.setup()
  const incrementHandler = jest.fn()
  const decrementHandler = jest.fn()
  render(
    <CounterTwo
      count={0}
      handleIncrement={incrementHandler}
      handleDecrement={decrementHandler}
    />
  )
  const incrementButton = screen.getByRole('button', { name: 'Increment' })
  const decrementButton = screen.getByRole('button', { name: 'Decrement' })
  await user.click(incrementButton)
  await user.click(decrementButton)
  expect(incrementHandler).toHaveBeenCalledTimes(1)
  expect(decrementHandler).toHaveBeenCalledTimes(1)
})

```

Mock HTTP requests

Consider this component

```

import { useState, useEffect } from 'react'

export const Users = () => {
  const [users, setUsers] = useState<string[]>([])
  const [error, setError] = useState<string | null>(null)
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((res) => res.json())
      .then((data) => setUsers(data.map((user: { name: string }) => user.name)))
      .catch(() => setError('Error fetching users'))
  }, [])
  return (
    <div>
      <h1>Users</h1>
      {error && <p>{error}</p>}
      <ul>
        {users.map((user) => (
          <li key={user}>{user}</li>
        ))}
      </ul>
    </div>
  )
}

```

Making requests to the real api every time can be expensive, so we mock APIs for testing using MSW (Mock Service Worker)

To set up MSW we install it , create a mock folder , create a server folder inside mock folder

server folder

```

// src/mocks/server.js
import { setupServer } from 'msw/node'
import { handlers } from './handlers'

```

```
// This configures a request mocking server with the given request handlers.  
export const server = setupServer(...handlers)
```

handler code

```
import { rest } from 'msw'  
  
export const handlers = [  
  rest.get('https://jsonplaceholder.typicode.com/users', (req, res, ctx) => {  
    return res(  
      ctx.status(200),  
      ctx.json([  
        {  
          name: 'Bruce Wayne',  
        },  
        {  
          name: 'Clark Kent',  
        },  
        {  
          name: 'Princess Diana',  
        },  
      ])  
    )  
  }  
),  
]  
]
```

```
import { render, screen } from '@testing-library/react'  
import { Users } from './Users'  
import { rest } from 'msw'  
import { server } from '../mocks/server'  
  
describe('Users', () => {  
  test('renders correctly', () => {  
    render(<Users />)  
    const textElement = screen.getByText('Users')
```

```

    expect(textElement).toBeInTheDocument()
  })

//without setup
//this will throw error because it will call real api with list length 10 not 3
test('renders a list of users', async () => {
  render(<Users />)
  const users = await screen.findAllByRole('listitem')
  expect(users).toHaveLength(3)
})

test('renders error', async () => {
  server.use(
    rest.get(
      'https://jsonplaceholder.typicode.com/users',
      (req, res, ctx) => {
        return res(ctx.status(500))
      }
    )
  )
  render(<Users />)
  const error = await screen.findByText('Error fetching users')
  expect(error).toBeInTheDocument()
})
})

```

setup

```

// jest-dom adds custom jest matchers for asserting on DOM nodes.
// allows you to do things like:
// expect(element).toHaveTextContent(/react/i)
// learn more: https://github.com/testing-library/jest-dom
import '@testing-library/jest-dom'
// src/setupTests.js
import { server } from './mocks/server'
// Establish API mocking before all tests.

```

```
beforeAll(() => server.listen())
```

```
// Reset any request handlers that we may add during the tests,  
// so they don't affect other tests.
```

```
afterEach(() => server.resetHandlers())
```

```
// Clean up after the tests are finished.
```

```
afterAll(() => server.close())
```